

# Aula 16 - BSTA

## Problema: Adivinhação

Imagine que você está brincando com um amigo seu de um jogo muito específico: o seu amigo vai pensar em um número inteiro qualquer  $x \in [1, 1000]$ . A sua tarefa será adivinhar esse número. Você vai poder fazer, no máximo, 10 palpites. Para cada palpite que você testar, seu amigo responderá se seu palpite é maior, menor ou igual a  $x$  (sendo que, se for igual a  $x$ , você ganha o jogo).

Faça um algoritmo interativo que simula esse problema e garante sempre adivinhar o número secreto dentro de 10 palpites.

De cara, essa ideia de “maior, menor ou igual” ditar a nossa próxima escolha, já sugere fortemente uma busca binária. A relação do tamanho do intervalo de busca com o número máximo de passos também é suspeita ( $2^{10} \approx 1000$ ).

E, de fato, essa é a solução. Nós atualizamos o intervalo de busca conforme o nosso amigo nos dá respostas, até convergir para o nosso alvo.

Esse é um caso básico de **BSAT**, ou “Binary Search On The Answer”. Nós aplicamos o algoritmo da busca binária não em um vetor de elementos como estamos acostumados, mas sim, no intervalo completo de respostas que possuímos.

## Problema: Um Desafio Muito Distinto

João e Maria estão brincando de um jogo. João, inicialmente, tem uma caixa vazia. Maria possui, em uma mesa, infinitas bolinhas de gude. A cada rodada do jogo, João pode pegar qualquer número inteiro  $x$  de bolinhas, tal que  $A \leq x \leq B$ , sendo  $A, B$  dois inteiros quaisquer tal que  $A \leq B$ , e posicioná-las dentro da sua caixa. A caixa de João tem capacidade máxima para  $L$  bolinhas. Existe, ainda, uma restrição extra: se João pegou  $x_1$  bolinhas na rodada atual, em todas as próximas rodadas, João deverá pegar um número diferente de  $x_1$ . Ou seja, em toda rodada, João deve pegar um número único de bolinhas.

A grande ideia do jogo consiste em uma aposta que Maria fez com João: para cada rodada que João conseguir jogar (antes da caixa encher de bolinhas de gude) ele ganha um chocolate. Dados os inteiros  $A, B, L$ , e visando maximizar o número de chocolates que João ganhará, faça um programa que diz quantos chocolates exatamente João vai ganhar.

## Ideia ingênua

Fica claro aqui que a melhor estratégia para maximizar o número de rodadas que João joga antes de sua caixa encher é sempre pegar o menor número de bolinhas que ainda podemos pegar. Ou seja, começamos em  $A$  bolinhas (o limite inferior) e seguimos pegando  $A + 1, A + 2, \dots, B$  bolinhas.

Sendo que, se a soma das bolinhas que pegamos desde a primeira rodada ultrapassar  $L$ , nós encerramos o processo e temos a resposta de quantas rodadas conseguimos jogar.

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int a, b, l;
    cin >> a >> b >> l;

    int soma = 0;
    for(int i = a; i < b; ++i){
        soma += i;
        if(soma >= l){
            cout << i - a + 1; // número de rodadas percorridas até agora
            return 0;
        }
    }
}
```

```
    cout << b - a + 1; // caso chegamos ao fim do for sem ultrapassar l
}
```

Esse código, teoricamente, já é muito bom (complexidade linear), mas, com a ideia de BSAT, conseguimos melhorar isso.

Aqui, o nosso intervalo de busca é  $[A, B]$ , e procuramos por um inteiro  $m \in [A, B]$  tal que

$$\sum_{i=A}^{m+1} i > l$$

Sendo que

$$\sum_{i=A}^m i \leq l$$

Ou seja, buscamos exatamente o ponto de virada. Uma forma de enxergar, é que temos um intervalo de respostas:

| Inteiro $m$                        | A   | A+1 | A+2 | A+3 | ... | B-1 | B   |
|------------------------------------|-----|-----|-----|-----|-----|-----|-----|
| Soma de A até $m$ passa do limite? | Não | Não | Não | Sim | Sim | Sim | Sim |

Dado que todas as respostas antes de um determinado número são válidas (a soma dos números não ultrapassa o limite), nós buscamos a MAIOR dessas respostas válidas.

A ideia para achar esse ponto de virada é, justamente, a BSAT. Se a soma estiver ultrapassando nosso limite, buscamos diminuir essa soma. Caso contrário, buscamos aumentar essa soma. Paramos quando o intervalo de busca convergir:

```
#include <bits/stdc++.h>
using namespace std;

long long soma_entre(long long x, long long y) {
    if(x > y) return 0;
    return (x + y) * (y - x + 1) / 2;
}

int main(){
    long long a, b, l, m;
    cin >> l >> a >> b;

    if(soma_entre(a, b) <= l){
        cout << b - a + 1 << "\n";
        continue;
    } // Caso em que podemos jogar TODAS as jogadas possíveis

    long long hi = b, lo = a; // Limites da BSAT
    long long ans = a; // Resposta inicial placeholder

    while(hi >= lo){
        m = (hi + lo) / 2;

        long long s = soma_entre(a, m - 1);

        if(s < l){
```

```

        ans = m;
        lo = m + 1;
    }
    else hi = m - 1;
}

cout << ans - a + 1 << "\n";
}

```

## Busca Binária na Resposta

Agora, abstraindo de um problema específico para a estratégia de fato, quando temos um problema que possui um intervalo de possíveis respostas ordenado e bem definido, como é o caso da adivinhação (a resposta está entre 1 e 1000) ou do desafio distinto (está entre A e B), sendo que, dessas respostas possíveis, uma parte delas é válida (correta), e a partir de certo ponto todas as respostas tornam-se inválidas, normalmente queremos justamente o ponto de virada (o maior/menor valor que satisfaz o problema).

Visualmente, quando temos algo como:

respostas: {O, O, O, O, O, O, X, X, X, X}

Ou:

respostas: {X, X, X, X, O, O, O, O, O}

Os X's representam respostas inválidas, e os O's representam respostas válidas. Quando queremos justamente o ponto de virada (o último O antes de um X ou o primeiro O depois de um X), caberia uma busca linear, sim, mas podemos otimizar com uma busca binária no intervalo de respostas, algo no formato:

```

int hi = limite_superior;
int lo = limite_inferior;
int ans = pior_possivel_por_enquanto;

while(hi >= lo){
    int m = (hi + lo)/2;

    if(testar(m)){
        ans = m;
        hi = m-1;
    }
    else lo = m+1;
}

```

Atentando-se sempre para o fato de que, dependendo de como o problema é estruturado (se queremos a “primeira resposta válida” ou a “última resposta válida”, quando um palpite  $m$  for válido, atualizamos para um subintervalo de busca diferente.

No caso do desafio, como queremos a última resposta válida, ao achar uma resposta válida qualquer, tentamos buscar uma resposta válida **maior** que a atual (então, buscamos no subintervalo da direita).

Um problema muito famoso dentro dessa estratégia é o “Pão a metro”, da OBI, disponível no Neps. Deem uma olhada nele.